



Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Michael Roman Alavez Colli

Fecha:
29 de Junio 2026

Introducción

En el desarrollo de software es común que varias personas trabajen de manera simultánea sobre un mismo proyecto. Esta forma de trabajo permite avanzar más rápido, pero también puede generar conflictos cuando no existe una adecuada organización del código fuente. La pérdida de información, la sobreescritura de archivos y la dificultad para identificar los cambios realizados son algunos de los problemas más frecuentes cuando no se utiliza un sistema de control de versiones.

Actualmente, herramientas como Git se han convertido en un estándar dentro de la industria del software, ya que permiten administrar de forma segura las modificaciones realizadas por cada desarrollador, mantener un historial de cambios y facilitar el trabajo colaborativo mediante el uso de ramas independientes.

En esta actividad se analiza el caso del proyecto "SmartLibrary", identificando los errores cometidos por el equipo de desarrollo y proponiendo una estrategia de organización utilizando Git. Además, se explica la importancia del control de versiones, se presenta un flujo de trabajo recomendado y se responden diversas situaciones que pueden presentarse durante el desarrollo colaborativo de un sistema.

Parte 1. Análisis del problema

1. ¿Cuáles fueron los principales errores cometidos por el equipo durante el desarrollo?

El principal error fue trabajar compartiendo únicamente una carpeta en la nube, donde cada integrante reemplazaba el proyecto completo con su propia versión. Esto ocasionó pérdida de cambios, sobreescritura de archivos y falta de control sobre las modificaciones realizadas. Además, no existía una organización para integrar el trabajo del equipo ni un registro de quién realizaba cada cambio.

2. ¿Por qué trabajar únicamente compartiendo carpetas genera tantos problemas?

Porque cada desarrollador trabaja sobre una copia diferente del proyecto y al subir nuevamente la carpeta puede reemplazar el trabajo realizado por otros compañeros. Además, no existe un historial de versiones ni herramientas para detectar conflictos entre archivos modificados.

3. ¿Qué consecuencias podría tener esta forma de trabajo en proyectos grandes?

En proyectos grandes podrían perderse semanas de trabajo, aumentar considerablemente los errores, retrasar las entregas, dificultar el mantenimiento del sistema y generar conflictos entre los integrantes del equipo debido a la falta de control sobre las modificaciones realizadas.

Parte 2. Importancia del Control de Versiones

¿Qué es un sistema de control de versiones?

Es una herramienta que permite registrar todos los cambios realizados en los archivos de un proyecto, facilitando el trabajo colaborativo y el seguimiento de las modificaciones realizadas por cada desarrollador.

¿Cuál es su principal objetivo?

Su principal objetivo es administrar de forma segura las diferentes versiones del proyecto, evitando la pérdida de información y permitiendo que varias personas trabajen al mismo tiempo sin afectar el trabajo de los demás.

¿Qué ventajas ofrece frente al manejo tradicional de archivos?

- Mantiene un historial completo de cambios.
- Permite recuperar versiones anteriores.
- Facilita el trabajo colaborativo.
- Evita sobrescribir archivos importantes.
- Identifica quién realizó cada modificación.
- Permite integrar cambios de manera controlada.
- Reduce errores durante el desarrollo.

Parte 3. Propuesta de organización

¿Qué rama utilizarías para la versión estable del sistema?

Utilizaría la rama **main**, ya que debe contener únicamente el código estable, probado y listo para producción.

¿Qué rama utilizarías para integrar el trabajo del equipo?

Utilizaría la rama **prueba**, donde se integrarían las funcionalidades desarrolladas por todos los integrantes antes de enviarlas a la rama principal.

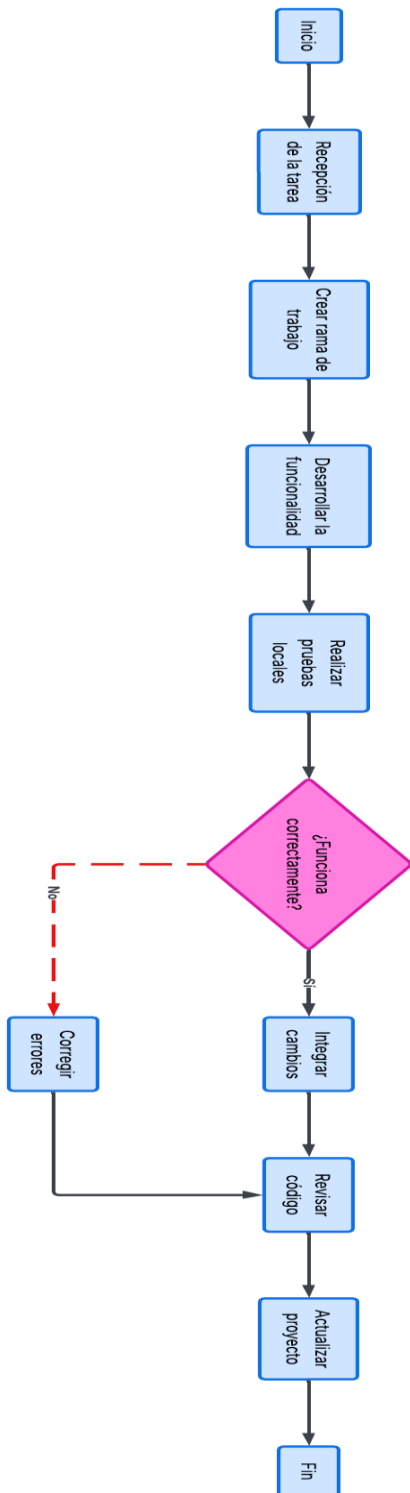
¿Cada desarrollador debería trabajar directamente sobre la rama principal?

No. Cada desarrollador debe crear su propia rama de trabajo para desarrollar nuevas funcionalidades o corregir errores. De esta forma se evita afectar la versión estable del proyecto y se pueden revisar los cambios antes de integrarlos.

¿Cuándo deberían integrarse los cambios al proyecto?

Los cambios deben integrarse únicamente después de haber sido probados, revisados y comprobado que funcionan correctamente. Así se reduce la posibilidad de introducir errores al sistema.

Parte 4. Diagrama de Flujo



Parte 5. Análisis de situaciones

1. ¿Qué ocurriría si dos desarrolladores modifican el mismo archivo simultáneamente?

Se produciría un conflicto de versiones. Git detecta esta situación y solicita al equipo decidir qué cambios conservar antes de integrar el código.

2. ¿Por qué es importante conservar un historial de cambios en un proyecto?

Porque permite conocer quién realizó cada modificación, cuándo se hizo y cuál fue el motivo. Además, facilita regresar a una versión anterior cuando aparece algún problema.

3. ¿Qué beneficios aporta trabajar mediante ramas independientes?

Permite que cada desarrollador trabaje de forma independiente, evita afectar el código principal, facilita las pruebas y reduce los conflictos durante la integración del proyecto.

4. ¿Cómo ayuda el control de versiones cuando aparece un error después de una actualización?

Permite identificar rápidamente el cambio que provocó el problema y regresar a una versión anterior mientras se corrige el error.

5. ¿Por qué el control de versiones es indispensable en proyectos desarrollados por varios programadores?

Porque organiza el trabajo colaborativo, evita pérdidas de información, mantiene un historial de cambios, facilita la integración del código y mejora la calidad del software desarrollado.

Conclusión

El desarrollo de esta actividad me permitió comprender la importancia del control de versiones dentro de los proyectos de software. Se pudo observar que trabajar únicamente compartiendo carpetas genera muchos problemas, como la pérdida de información, la sobreescritura de archivos y la dificultad para identificar los cambios realizados por cada integrante del equipo.

También se comprendió que Git se ha convertido en un estándar de la industria debido a que ofrece herramientas que facilitan el trabajo colaborativo, mantienen un historial completo de modificaciones y permiten organizar el desarrollo mediante ramas independientes. Gracias a estas características es posible integrar cambios de forma segura y reducir considerablemente los errores durante el desarrollo.